

# Toward a Theory of Computation for Learned Transformer Machines: Tasks, Resources, Learnability, Composition, and Closure

Paper 0.1 — Position and Formal Framework

Draft inception — August 2026

## Abstract

The dominant theoretical question about large language models is often phrased too coarsely: “what can a Transformer compute?” Classical expressivity results characterize idealized Transformers through formal languages and circuit classes, while empirical work usually asks whether a trained model succeeds on a benchmark. Between these views lies a missing object: the computational theory of a *learned* Transformer machine. A computation may be representable by an architecture yet rarely acquired by training; it may be learned on bounded instances yet fail to compose or close over instance depth. Moreover, the computational object in a modern application is usually compound:

$$\mathcal{M} = (F_\theta, C, S, I, \mathbb{T}, \mathcal{B}),$$

combining a learned controller, execution protocol, writable state, information interfaces, a tool algebra, and finite resource budgets. These are product-space coordinates, not a scalar “agent level.” We distinguish information sufficiency, representability, acquisition, execution reliability, efficiency, systematic generalization, and closure. Tasks are classified by required state and iteration; experiments estimate competence and acquisition frontiers rather than infer algorithms from endpoint accuracy. Papers 0.5–0.9 provide later empirical anchors, but none is required for the framework.

## 1 Why a learned-machine theory is needed

A fixed architecture can represent computations that gradient-based training rarely discovers. A trained model can fit finite examples without acquiring a reusable rule. A rule can be reusable over a bounded range without supporting closure. Conversely, autoregressive generation can repeatedly apply the same learned local transformation and thereby change the effective computational machine without changing the model weights.

We therefore separate:

$$\text{representability} \neq \text{learnability} \neq \text{reliable acquisition} \neq \text{systematic executability} \neq \text{closure}.$$

Classical Transformer theory provides essential upper and lower bounds under explicit architectural and precision assumptions. Formal-language work relates fixed-depth attention models to circuit classes such as  $AC^0$  and  $TC^0$ , while recent chain-of-thought theory shows that autoregressive intermediate computation can substantially expand expressivity. Our objective is complementary: characterize the *trained finite machine* and measure how its empirical computational frontier depends on architecture, training distribution, generated state, context, and external execution resources.

## 2 Basic objects

### 2.1 Task family

A task family is

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, \mathcal{D}, \mathcal{R}),$$

where  $\mathcal{D}$  is a generator/distribution and  $\mathcal{R}$  is the declared structural prior that makes prediction possible.

Each instance has a complexity vector

$$\mathbf{C} = (S, K, B, V, N, M, \dots),$$

where  $S$  is atomic syntax/operator complexity,  $K$  required composition/proof depth,  $B$  branching/search width,  $V$  variable/state complexity,  $N$  nuisance context, and  $M$  memory/context requirement. Complexity is generally partially ordered, so measured Pareto competence regions are preferable to a single scalar difficulty.

## 2.2 Information sufficiency

Before studying an intelligent mechanism, determine whether the task is information-theoretically solvable. For query  $X$ , context  $D$ , and target  $Y$ :

$$I(Y; D \mid X) = H(Y \mid X) - H(Y \mid X, D).$$

For deterministic rule inference under declared prior  $\mathcal{R}$ , require

$$H(Y \mid X, D, \mathcal{R}) = 0.$$

An arbitrary unseen random mapping with insufficient demonstrations is not a reasoning problem: no oracle lacking additional information can exceed the Bayes ceiling.

Every task release should therefore include an information-validity record

$$\mathcal{I} = (H(Y \mid X), H(Y \mid X, D), I(Y; D \mid X), \mathcal{R}, A_{\text{Bayes}}, |\mathcal{H}_{X,D,\mathcal{R}}|, m_{\text{id}}),$$

where  $\mathcal{H}_{X,D,\mathcal{R}}$  is the set of hypotheses consistent with the query, demonstrations, and declared prior, and  $m_{\text{id}}$  is the minimum context needed for identifiability. Entropies are distribution-dependent quantities, not properties of a serialization alone. If the generator is deterministic only after conditioning on  $\mathcal{R}$ , that conditioning must be stated rather than smuggled into an oracle.

**Proposition 1** (Information impossibility). *Under zero-one loss, if  $H(Y \mid X, D, \mathcal{R}) > 0$ , then no predictor measurable with respect to  $(X, D, \mathcal{R})$  is universally correct. Its optimal expected accuracy is*

$$A_{\text{Bayes}} = \mathbb{E}_{X,D,\mathcal{R}} \max_y P(Y = y \mid X, D, \mathcal{R}) < 1$$

*whenever the conditional distribution is non-degenerate on a set of positive measure.*

Pointwise, the best decision selects the most probable conditional label; positive conditional entropy entails ambiguity on at least one event of positive measure. This separates model failure from task underdetermination.

## 2.3 Controller hierarchy and compound machines

The controller coordinate has four operational levels. At  $C_0$ , a fixed pass computes  $Y = F_\theta(X)$ . At  $C_1$ , autoregression computes  $y_t = F_\theta(X, y_{<t})$ ; generated tokens form serialized writable state. At  $C_2$ , the state language is structured into fields such as CURRENT, TARGET, COUNTER, STACK, SUBSTITUTION, or PROOF-STATE. At  $C_3$ , an external loop repeatedly inspects state, selects an action or tool, incorporates its result, and decides whether to continue. A practical “agent” is typically a  $C_3$  protocol, not automatically a new formal class.

Controller sophistication is only one coordinate. We define the compound learned machine

**Definition 1** (Compound learned machine).

$$\mathcal{M} = (F_\theta, C, S, I, \mathbb{T}, \mathcal{B}),$$

*where  $F_\theta$  is the learned Transformer/controller;  $C$  is its execution protocol;  $S$  is the writable-state interface;  $I : q \mapsto D$  supplies external information;  $\mathbb{T} = \{T_1, \dots, T_m\}$  is a set of partial external operations  $T_i : \mathcal{X}_i \rightarrow \mathcal{Y}_i$ ; and  $\mathcal{B}$  bounds context, calls, tokens, time, and memory. For acquisition process  $\Pi$ ,*

$$A(\tau, \mathcal{C}; F_\theta, C, S, I, \mathbb{T}, \mathcal{B}, \Pi), \quad P_{\text{acquire}} = P_\Pi[A \geq \gamma].$$

This product description replaces a scalar tool/agent ordering. Autoregression and structured traces alter  $C$  and  $S$ ; retrieval primarily alters  $I$ ; a local graph edge, a closure oracle, and Python alter  $\mathbb{T}$  in categorically different ways. Every comparison should record

$$\Delta \mathcal{M} = (\Delta C, \Delta S, \Delta I, \Delta \mathbb{T}, \Delta \mathcal{B}, \Delta \Pi),$$

so an easier prompt, a stronger primitive, more execution time, and better training are not all called “augmentation.”

### 3 Six computational axes

#### 3.1 Axis 1: atomic syntax and local operator learning

What is the most complex single-step relation a model of size  $(L, W, H)$  learns reliably? A rough ladder is

$$\begin{aligned} &\text{association} \rightarrow \text{n-gram} \rightarrow \text{unary} \rightarrow \text{binary} \rightarrow \text{Boolean rule} \\ &\rightarrow \text{predicate+variables} \rightarrow \text{nested functor} \rightarrow \text{type dispatch} \rightarrow \text{rewrite}. \end{aligned}$$

Define acquisition probability

$$P_{\text{acquire}}(\tau; M, \mathcal{D}, T) = P_s[A_s(\tau) \geq \gamma]$$

and coverage over the intended legal instance set. The falsifiable resource question is whether a controlled change in width, heads, layers, data, or budget moves this boundary across seeds. A single successful seed establishes neither learnability nor reliable acquisition.

#### 3.2 Axis 2: internal one-pass composition

Assume local operator  $f$  is competent. How much composition can one forward pass implement?

$$x \mapsto f(x) \mapsto f^2(x) \mapsto \dots \mapsto f^K(x).$$

Define

$$K_{\text{internal}}(M, \tau) = \max\{K : A(f^K; M) \geq \gamma\}.$$

A larger frontier does not prove serial layer-by-layer execution. Required dependency radius, operator count, proof depth, branching, nuisance, and raw length must be crossed rather than conflated. Ordered intermediate decoding and causal transfer are required before an  $L \times K$  interaction is interpreted as serial internal computation.

#### 3.3 Axis 3: autoregressive multi-token composition

Autoregression exposes recurrence:

$$s_{t+1} = F_\theta(X, s_{\leq t}).$$

For a parent relation, a root trajectory may be serialized as

$$C \rightarrow B \rightarrow A \rightarrow R.$$

Self-attention can read the entire generated history, so the state may be a tuple, proof prefix, substitution, or counter. Define  $K_{\text{AR}}$  and compare it directly with  $K_{\text{internal}}$ . Report local transition accuracy, stopping accuracy, total-sequence success, generated steps, and train-short/test-long behavior. This distinguishes a competent transition with accumulated error from a failed recurrence controller.

#### 3.4 Axis 4: closure

Distinguish:

$$\forall K < \infty \exists M_K$$

(bounded family solvability), from a fixed-model tested range

$$\exists M \forall K \leq K_{\text{max}}^{\text{test}},$$

and from true algorithmic closure

$$\exists M \forall K < \infty.$$

The last cannot be established empirically, but closure signatures can be tested by train-short/test-long extrapolation, computation length scaling with instance complexity, local-transition reuse, and termination generalization. Finite context creates a separate practical memory boundary.

**Definition 2** (Budgeted closure signature). *For a fixed trained machine  $m$  and explicit context/step budget  $C_{\text{max}}$ , define*

$$\Sigma_{m, C_{\text{max}}} = (K_{\text{train}}, K_{\text{test}}, K_{\text{pass}}, \rho_{\text{step}}, \rho_{\text{stop}}, g(K), f_{\text{fail}}),$$

where  $K_{\text{pass}}$  is the largest tested complexity passing the multi-seed gate,  $\rho_{\text{step}}$  and  $\rho_{\text{stop}}$  are local-transition and stopping reliability,  $g(K)$  is observed computation-length scaling, and  $f_{\text{fail}}$  is the first classified failure. This is a finite empirical object, not algorithmic closure.

### 3.5 Axis 5: in-context information

ICL changes what facts, rules, or parameters are available without changing weights. Separate fact acquisition from procedure acquisition. Contextual tasks should be classified as copy, constraint completion, finite task selection, parametric rule induction, or compositional/procedural induction. Holding  $X$  fixed while varying  $D$  is essential. Even when  $I(Y; D \mid X) > 0$ , controls must distinguish copying, elimination, task selection, parameter estimation, and construction of a reusable procedure.

### 3.6 Axis 6: harness and tool augmentation

External execution may expand representable computation, improve acquisition, stabilize execution, or merely save resources. These are different claims. A proper factorial comparison crosses controller  $C_0$ – $C_3$  with tool semantics rather than placing “tools” above autoregression on one ladder. Without controller-matched and tool-matched baselines, delegated computation can be mistaken for neural acquisition.

## 4 Information interfaces and tool algebra

An information interface  $I(q) = D$  changes the evidence available to the controller. Its characteristic contribution is a conditional-entropy reduction  $H(Y \mid X, D) < H(Y \mid X)$ , not necessarily a stronger inference procedure. Basic retrieval-augmented generation is therefore approximated by

$$q \mapsto I(q) = D \mapsto F_\theta(q, D),$$

or “external information selection plus in-context computation.” Ranking, query decomposition, iterative search, and filtering may add control or computation, but must be classified separately from retrieval itself.

The tool coordinate is semantic. We use:  $T_0$ , information lookup such as **retrieve** or **read**;  $T_1$ , a local transition such as **parent**, **neighbor**, or **successor**;  $T_2$ , a deterministic transformation such as arithmetic, sorting, simplification, or unification;  $T_3$ , search or closure such as **root**, **shortestPath**, or recursive SQL; and  $T_4$ , a programmable executor such as Python, shell, SQL, or Cypher. A single  $T_2$  or  $T_3$  call may hide many elementary steps. A  $T_4$  interpreter may be universal under suitable language and resource assumptions, yet this says nothing about learned program synthesis, reliability, description length, or efficiency.

Let  $\langle \mathbb{T} \rangle_C^g$  denote computations obtainable by finite, budgeted composition of  $\mathbb{T}$  under controller  $C$ . A  $C_3$  controller with **parent** can represent repeated **parent**<sup>k</sup> if it also represents state update and termination. Supplying **root** instead moves the task-defining closure inside the tool. Supplying an interpreter  $T_{\text{int}}(p, x) = p(x)$  moves the learned problem further toward program synthesis and call reliability. Computability equivalence between two interfaces therefore does not imply equal acquisition probability or cost.

| Tool/information            | $C_0$ fixed pass                       | $C_1$ autoregressive                             | $C_2$ structured state                           | $C_3$ iterative control                                  |
|-----------------------------|----------------------------------------|--------------------------------------------------|--------------------------------------------------|----------------------------------------------------------|
| None                        | one-shot prediction                    | ordinary generation; CoT                         | typed scratchpad                                 | repeated model/environment loop                          |
| $T_0$ lookup<br>$T_1$ local | basic RAG context<br>one external step | retrieve then generate<br>serialized local calls | cited/evidence state<br>explicit current/counter | iterative RAG<br>GraphRAG traversal;<br>parent iteration |
| $T_2$ transform             | delegated transform                    | calculator/PAL call                              | typed expression/result                          | CAS or nonrecursive SQL<br>agent                         |
| $T_3$ closure               | delegated answer                       | closure API call                                 | explicit query/result                            | shortest-path or<br>recursive-query agent                |
| $T_4$ interpreter           | execute supplied program               | generate then execute                            | program plus execution<br>state                  | code/SQL/Cypher agent                                    |

Table 1: Controller  $\times$  tool/information product space. Entries are examples, not formal equivalence classes; state, budget, and training remain separate coordinates.

### 4.1 Delegation and four kinds of gain

For benchmark function  $f$ , a tool pipeline can be written conceptually as

$$f = g \circ T \circ h,$$

where  $h$  serializes arguments,  $T$  performs an external operation, and  $g$  integrates the result. With  $T = \text{parent}$ , **root** finding still requires controller-side iteration and termination. With  $T = \text{root}$ , the benchmark is directly delegated. Every tool result should identify where the task-defining computation occurs.

Four gains must be reported separately: *computational gain*, when a resource makes a computation representable under stated base-machine constraints; *acquisition gain*, when it raises  $P_{\text{acquire}}$  although both machines can represent the procedure; *execution-reliability gain*, when it raises  $P_{\text{exec}}(\text{success} \mid \text{acquired})$ ; and *efficiency gain*, when it reduces examples, generated steps, errors, latency, memory, or model size. Endpoint accuracy alone cannot identify which occurred.

As an accounting identity rather than an independence model,

$$\begin{aligned} P(\text{success}) &= P(\text{information sufficient}) \\ &\quad \times P(\text{procedure acquired} \mid \text{information}) \\ &\quad \times P(\text{execution succeeds} \mid \text{acquired, information}). \end{aligned}$$

Tool execution further factors into action selection, argument correctness, tool success, result integration, and termination. These conditional events are the natural failure taxonomy for Paper 0.9.

**Proposition 2** (Direct delegation). *If tool  $T$  computes benchmark target  $f$  exactly and a controller reliably serializes the input and interprets the output, success establishes acquisition of the call-and-integrate interface, not neural acquisition of  $f$ .*

The distinction is empirical: argument formation and result use may remain difficult, but the defining computation occurs in  $T$ .

**Proposition 3** (Budgeted composition of a local primitive). *If controller  $C$  reliably maintains state, invokes local primitive  $T$ , and applies a correct termination predicate, then  $T^k(x)$  is representable for every finite  $k$  whose trace fits  $\mathcal{B}$ .*

This is a representability statement, not a guarantee that  $\Pi$  acquires the controller or that errors remain bounded.

## 4.2 Common systems are compound configurations

RAG, GraphRAG, calculators, and agents are application patterns rather than single computational classes. GraphRAG is heterogeneous:  $\text{neighbor}(v)$  is  $T_1$ , while community aggregation is a substantial  $T_2$ -like transform. Shortest-path and common-ancestor APIs are  $T_3$ ; Cypher is a  $T_4$ -like programmable query interface. Microsoft GraphRAG combines graph construction, network analysis, retrieval, prompting, and summarization; its product name does not determine one row of the algebra [16].

| System                                     | Controller                  | Information                   | Tool algebra      | Added resource                            | Task-defining computation                              |
|--------------------------------------------|-----------------------------|-------------------------------|-------------------|-------------------------------------------|--------------------------------------------------------|
| One-shot / CoT<br>Basic / iterative<br>RAG | $C_0/C_1$<br>$C_0, C_1/C_3$ | prompt<br>retrieved documents | none<br>$T_0$     | pass / serial tokens<br>external evidence | learned controller<br>retrieval plus<br>in-context use |
| Local GraphRAG                             | $C_3$                       | graph                         | $T_1$ neighbor    | graph access, iteration                   | controller traversal                                   |
| Search GraphRAG                            | $C_3$                       | graph                         | $T_3$ path/search | delegated closure                         | graph engine                                           |
| Calculator / CAS                           | $C_1/C_3$                   | prompt                        | $T_2$             | arithmetic/symbolic<br>engine             | often external engine                                  |
| SQL agent                                  | $C_3$                       | database                      | $T_2/T_4$         | query language/engine                     | split; recursive SQL<br>can close                      |
| Code interpreter                           | $C_3$                       | files/state                   | $T_4$             | loops, recursion,<br>memory               | synthesized program +<br>runtime                       |
| ReAct / multi-tool                         | $C_3$                       | observations                  | mixed             | action-observation loop                   | depends on selected<br>primitive                       |

Table 2: System mapping. Calculator serialization, CAS operations, SQL planning, and code generation remain learned tasks even when execution is external.

## 5 Task classes by required state

| Class                  | Required computation           | Example          | Minimal resource in principle       | Empirically helpful resource   |
|------------------------|--------------------------------|------------------|-------------------------------------|--------------------------------|
| A: local               | $x \mapsto f(x)$               | parent           | $C_0$ or one $T_1$ call             | width/data or reliable call    |
| B: fixed composition   | $f^k(x)$                       | grandparent      | finite $C_0$ depth or bounded trace | $C_1/C_2$ state                |
| C: termination closure | iterate to predicate           | root             | recurrence + stop                   | $C_3 + T_1$ ; $T_3$ delegates  |
| D: variable count      | $(x, k) \mapsto (f(x), k - 1)$ | ancestor $_k$    | counter/state + recurrence          | $C_2/C_3 + T_1$                |
| E: multi-state         | update several pointers        | paired traversal | tuple state + recurrence            | typed $C_2$ fields             |
| F: search              | frontier/visited set           | proof search     | memory + branching control          | $C_3$ ; $T_3$ delegates search |
| G: binding             | environment $\sigma$           | unification      | substitution state                  | $C_2$ or $T_2$ unifier         |
| H: proof state         | repeated rule choice           | theorem proving  | structured state + control          | $C_3$ plus local rules         |

Table 3: Task-state progression. “Minimal” is a representability statement under explicit budgets; “helpful” is an empirical hypothesis, and closure/search tools may delegate the task.

## 6 A learned-computation phase diagram

For machine class  $\mathcal{M}$ , task family  $\tau$ , complexity  $\mathbf{C}$ , and training process  $\Pi$ , define

$$A(\tau, \mathbf{C}; \mathcal{M}, \Pi), \quad P_{\text{acquire}}(\tau, \mathbf{C}; \mathcal{M}, \Pi).$$

Here  $\Pi$  is a distribution over initialization  $s_\theta$ , minibatch/order  $s_b$ , generator  $s_g$ , and optimizer/hyperparameters  $h$ , not a single lucky run:

$$P_{\text{acquire}} = \Pr_{s_\theta, s_b, s_g, h} \{A \geq \gamma, A_{\text{ctrl}} < \gamma_{\text{ctrl}}\}.$$

For a tested domain  $\Omega_{\tau, \mathbf{C}}$ , coverage is the measured mass or fraction on which the gate holds. The internal and autoregressive frontiers  $K_{\text{internal}}$  and  $K_{\text{AR}}$  are task- and protocol-specific projections of a generally non-rectangular competence region. A complete report distinguishes Bayes solvability, representability, learnability, reliable acquisition, coverage, composition frontier, recurrence frontier, and closure signature.

Let  $Q$  be the latent event that a reusable procedure was acquired and  $S$  the event of evaluation success. Then

$$P(S) = P(Q)P(S | Q) + P(\neg Q)P(S | \neg Q).$$

The shorthand  $P(S) \approx P_{\text{acquire}}P(S | Q)$  is valid only when shortcut success in the  $\neg Q$  class is negligible. Because  $Q$  is not directly observed, controls and interventions identify a useful mixture model rather than a unique psychological state.

**Proposition 4** (Local-error accumulation). *If an execution requires  $T$  transitions and, conditional on every previously correct state, each transition is correct independently with probability  $1 - \epsilon$ , then exact trajectory success is  $(1 - \epsilon)^T \leq e^{-\epsilon T}$ .*

Independence is a diagnostic baseline, not a default model: correlated errors, recovery, verification, or history-dependent attention can produce slower or faster decay. Reporting both local and sequence accuracy tests which regime applies.

**Proposition 5** (Finite evidence does not establish closure). *For any finite tested set  $K \leq K_{\text{test}}$ , there exist a closed procedure and a bounded lookup machine that agree on every observation and diverge for some  $K > K_{\text{test}}$ .*

Construct the lookup machine to copy the closed procedure on the finite tested domain and return a fixed wrong answer outside it. Thus empirical evidence can support closure signatures and falsify particular bounded hypotheses, but cannot prove the universal quantifier.

No empirical resource monotonicity is assumed. Although a larger architecture may contain a smaller computation representationally, its optimizer, implicit bias, attention geometry, and finite-data basin differ. Hence  $P_{\text{acquire}}(L + 1) \geq P_{\text{acquire}}(L)$  is an empirical hypothesis, not an axiom.

Failure labels should include:

Bayes-impossible, missing fact, operator, composition, recurrence, counter/control, termination, search, context overflow, accumulated local error, acquisition-basin failure.

## 7 Relation to classical computation theory

Classical automata theory classifies languages and machines using memory, state transition, stack access, and recursion. Circuit complexity classifies computations by size, depth, fan-in, and gate type. The analogies below are conditional descriptions of resources, not identities between neural architectures and classical machines.

A  $C_0$  fixed Transformer is naturally compared with a bounded circuit only after specifying hard versus soft attention, numerical precision, positional encoding, uniformity, sequence-length scaling, and whether depth and size are fixed or grow.  $AC^0/TC^0$ -style statements established for one variant do not transfer automatically to another [1, 2, 10, 5]. A  $C_1$  autoregressive Transformer adds serial computational time through generated tokens; it is better described as a learned sequential transducer with readable serialized history than as a finite-state automaton absent an explicit finite-state restriction. Li et al.’s separation result makes this resource change precise under their assumptions [6].

A  $C_2$  scratchpad can serialize counters, stacks, substitutions, proof states, registers, or tape-like configurations. These analogies identify the state a task requires; they do not show that the architecture acquired a counter machine, pushdown automaton, register machine, or Turing machine. A  $C_3$  tool controller resembles an instruction-set- or oracle-augmented machine, but “oracle” suppresses real tool cost and failure. A  $T_4$  interpreter can provide general program execution under finite resources, making pure computability too coarse: learned program description, synthesis probability, execution reliability, and budget dominate practical behavior.

The present program adds an empirical learned-machine dimension. A construction proving that a Transformer *can* implement an operation establishes representability, not that SGD will find it from a finite generator. Turing completeness in principle does not imply reliable algorithm acquisition. Conversely, bounded empirical failure does not establish architectural impossibility.

The desired bridge is

machine resources + task structure + training distribution  
 $\longrightarrow$  acquisition and execution frontier

## 8 Related work

Table 4 locates the learned-machine layer relative to representative theory. The labels summarize theorem assumptions rather than asserting equivalence between models.

| Work                     | Machine                       | Assumptions emphasized                    | Result type                        | Gap addressed here                   |
|--------------------------|-------------------------------|-------------------------------------------|------------------------------------|--------------------------------------|
| Hahn (2020)              | fixed attention               | bounded depth;<br>formal-language setting | limitation/sensitivity             | training and finite acquisition      |
| Hao et al. (2022)        | fixed hard attention          | circuit assumptions, precision            | $AC^0$ -related bounds             | empirical competence regions         |
| Merrill et al. (2022)    | saturated attention           | asymptotic precision/size assumptions     | threshold-circuit characterization | finite learned frontier              |
| Pérez et al. (2021)      | recurrent/encoded Transformer | explicit precision/encoding assumptions   | Turing-completeness construction   | probability of learning construction |
| Strobl et al. (2024)     | multiple variants             | positions, precision, uniformity          | expressivity survey                | common reporting schema              |
| Li et al. (2024)         | autoregressive CoT            | serial intermediate tokens                | expressivity separation            | acquired finite $K_{AR}$             |
| Joshi et al. (2025)      | autoregressive learner        | time-invariant generator                  | learning/sample analysis           | seed and generator acquisition       |
| Bavandpour et al. (2025) | hard-attention CoT            | scratchpad assumptions                    | length lower bounds                | measured closure signatures          |
| Huang et al. (2025)      | trained recognizer            | regular-language distributions            | training dynamics                  | broader task/resource phase diagram  |

Table 4: Representative related-work matrix. Precision, positional assumptions, and uniformity must be read from each theorem; no row transfers automatically to the finite models studied here.

Formal-language and circuit-complexity analyses have established limitations and capabilities of fixed-depth Transformer variants under explicit assumptions [1, 2]. Strobl et al. survey this literature and emphasize that conclusions depend critically on precision, positional encodings, attention definitions, and uniformity [5]. Universality and Turing-completeness constructions establish strong representability under richer assumptions [4, 3], but do not answer finite-data acquisition probability.

Recent chain-of-thought theory is especially relevant. Li et al. show that intermediate autoregressive steps can enable inherently serial computations inaccessible to shallow one-pass models under their assumptions [6]. Joshi et al. formalize learning with an iterated time-invariant next-token generator and analyze sample/computational complexity [7]. Bavandpour et al. derive lower bounds on scratchpad length for algorithmic problems in hard-attention Transformers [8]. Huang et al. study training dynamics for regular-language recognition [9]. These works motivate treating generated tokens as a computational resource rather than merely explanatory prose.

Compound-system work supplies concrete configurations. Retrieval-augmented generation adds a learned retriever and retrieved evidence to generation [11]. MRKL routes queries among neural and symbolic modules [12]; Toolformer studies self-supervised acquisition of tool calls [13]; program-aided language models delegate execution of generated programs [14]; and ReAct interleaves learned reasoning traces, actions, and observations [15]. Their endpoints should be reanalyzed through Table 1: what information, state, primitive semantics, control, and budget changed, and was the observed gain computational, acquisitional, reliable, or efficient?

Mechanistic evidence answers another layer. Induction-head work demonstrates circuit emergence and links it to in-context behavior [17]; residual-stream decompositions provide a vocabulary for attributing component updates [18]; and causal tracing tests necessity by controlled activation replacement rather than by geometry alone [19]. Circuit existence, causal necessity, acquisition dynamics, systematic generalization, and length/closure extrapolation are progressively stronger and should not be collapsed into one claim of “understanding.”

Our emphasis is different: measure learned finite-machine frontiers, acquisition probability, information sufficiency, and transitions between one-pass, recurrent, contextual, and harnessed execution in tiny models where causal inspection is feasible.

## 9 Implications for Paper 0.9

Paper 0.9 should not compare an undifferentiated “model” with an “agent.” Its primary factorial design crosses controller—one-token  $C_0$ , free autoregression  $C_1$ , structured state  $C_2$ , iterative control  $C_3$ —with tool—none, local  $T_1$ , deterministic  $T_2$ , closure/search  $T_3$ , and programmable  $T_4$ . The first contrast should be none versus a local primitive such as `parent(x)`. That leaves iteration and termination with the controller and therefore tests recurrence acquisition. A `root(x)` or shortest-path API is an upper delegation control; it tests calling and integration, not learned closure.

For every trajectory Paper 0.9 should record information sufficiency, procedure acquisition, action selection, argument correctness, tool success, result integration, and termination. Conditional local accuracy and exact trajectory accuracy test the independent-error baseline; correlated failure and self-correction must be measured rather than assumed. Results should report controller-by-tool competence, call and token budgets, execution cost, and where the benchmark-defining computation occurs. This factorization connects failure modes to the compound machine instead of assigning them vaguely to “agency.”

## 10 Distributed cognition and heterogeneous computational intelligence

The compound-machine formalism admits a broader, secondary interpretation: useful computation need not be localized in one model. Distributed cognition has long analyzed cognition at the level of people, representations, artifacts, and environment [20]. This systems stance is related to, but weaker than, the extended-mind thesis of Clark and Chalmers [21]. Nothing here requires a metaphysical claim that a database, API, or notebook literally constitutes a person’s mind. The claim is operational: heterogeneous components can participate in one stateful computation, and the location of task-relevant work can be measured across their interfaces.

### 10.1 A heterogeneous computational graph

Extend a compound machine to a directed graph

$$\mathcal{G} = (V, E), \quad N_i : (X_i, S_i) \longrightarrow (Y_i, S'_i), \quad v_i \in V,$$



where an edge  $e_{ij} : Y_i \rightarrow X_j$  is a communication channel or callable interface. Nodes may be learned models, humans, calculators, databases, retrievers, graph engines, theorem provers, interpreters, APIs, or microservices. This shared node notation asserts interface participation, not internal equivalence. A human and a deterministic service differ radically in adaptation, grounding, memory, failure, and mechanism even when both accept a request and return a result.

For each node, retain a vector rather than an “intelligence score”:

$$\chi_i = (\mathcal{F}_i, c_i, \ell_i, r_i, a_i, m_i, \kappa_i),$$

where  $\mathcal{F}_i$  is its supported function family,  $c_i$  cost,  $\ell_i$  latency,  $r_i$  reliability,  $a_i$  adaptability/generalization,  $m_i$  state capacity, and  $\kappa_i$  its iteration or closure characteristics. A calculator is narrow and exact; an LLM is broad and adaptive but probabilistic; a database exposes a narrow query interface over large persistent state; a human is broadly adaptive and grounded but slow, costly, and working-memory limited. The vector makes complementarity visible without erasing mechanism.

For goal  $g$ , a controller selects a possibly branching, stateful trajectory

$$\pi_g = (N_{i_1}, N_{i_2}, \dots, N_{i_T}), \quad A(g; \mathcal{G}, \pi_g, \mathcal{B}).$$

Writing  $F_{\pi_g} = N_{i_T} \circ \dots \circ N_{i_1}$  is useful shorthand, but real trajectories exchange messages, revisit nodes, branch on observations, and mutate state. Routing is itself a learned or programmed problem. A cautious design objective is to choose  $\pi$  under accuracy, cost, latency, reliability, and resource constraints; no single scalar utility is assumed here.

An orchestrating model need not implement every sub-operation internally, but it must recognize a subproblem, select a node, parameterize its interface, integrate the result, verify it when needed, update state, and continue or terminate. Corresponding conditional quantities include

$$P_{\text{select}}, \quad P_{\text{parameterize}}, \quad P_{\text{integrate}}, \quad P_{\text{verify}}, \quad P_{\text{continue}}.$$

A perfect solver is useless behind an interface the controller cannot invoke. These terms refine the earlier decomposition into action choice, argument formation, and result integration. They make orchestration an empirical capability rather than a synonym for intelligence.

## 10.2 Reachable composition and distributed closure

Network capability is neither the maximum nor the simple union of node capabilities:

$$\mathcal{F}_{\mathcal{G}} \neq \bigcup_i \mathcal{F}_i, \quad \mathcal{F}_{\mathcal{G}} \subseteq \langle \mathcal{F}_1, \dots, \mathcal{F}_n \rangle_{\text{routing, state, budget}}.$$

The left inequality allows a routed composition that no node realizes alone; the right inclusion emphasizes that inaccessible interfaces, bad serialization, missing state, unreliable routing, or insufficient budget can prevent even individually available operations from composing. This is exactly the controller–state–information–tool algebra lifted from one compound machine to a network: each node exposes a local transition or compound submachine, while edges and routing determine reachable compositions.

A component may have only a finite measured frontier  $K_i$ , while the network exhibits a larger *system-level* or *distributed closure signature*. For example, an LLM that cannot reliably execute  $f^{1000}$  internally may synthesize a loop and invoke a  $T_4$  interpreter. Then  $K_{\text{LLM}} < K_{\text{LLM}+\text{interpreter}}$  over the measured budget. The closure belongs to the routed system, and the LLM’s learned contribution may be program synthesis and result integration rather than iteration. As before, finite context, calls, time, memory, and interpreter limits prohibit an unqualified claim of mathematical closure.

## 10.3 Real systems and non-equivalent nodes

Humans have long externalized memory and computation into speech, writing, notation, books, maps, calculators, computers, and networks. This is not a strict evolutionary ladder; it is a recurring pattern of persistent representation and specialized delegation. LLM-centered systems add a potentially useful node type: an adaptive semantic interface and probabilistic controller between rigid typed services. This flexibility complements, rather than dominates, the high reliability of narrow deterministic components.

| System              | Representative nodes/route                                             | Distributed contribution                             | Principal failure boundary                |
|---------------------|------------------------------------------------------------------------|------------------------------------------------------|-------------------------------------------|
| RAG assistant       | human $\rightarrow$ LLM $\leftrightarrow$ retriever/store              | evidence access plus semantic integration            | retrieval sufficiency, citation/use       |
| Graph assistant     | LLM $\leftrightarrow$ graph store/API                                  | local traversal or delegated search by API semantics | query choice, closure delegation          |
| Calculator/CAS      | LLM $\rightarrow$ arithmetic/symbolic engine $\rightarrow$ LLM         | semantic dispatch plus exact transform               | serialization and interpretation          |
| Coding agent        | human $\rightarrow$ LLM $\rightarrow$ editor/compiler/tests/repository | deterministic feedback on generated actions          | patch validity, test coverage, stopping   |
| Scientific workflow | human, LLM, Python, simulator, database, statistics                    | heterogeneous modeling and verification loop         | model choice, data validity, review       |
| Enterprise workflow | human, LLM, APIs, SQL, business rules                                  | semantic routing among typed services                | authorization, schema, policy, escalation |

Table 5: Examples of heterogeneous goal execution. No row implies that humans, learned models, and services are mechanistically or cognitively equivalent.

The system abstraction can make it less practically important that one node internally realize every sub-operation, but it makes component science more important, not less: system capability depends on the actual frontier, calibration, and failure distribution of each node. A trajectory such as human  $\rightarrow$  coding model  $\rightarrow$  compiler  $\rightarrow$  tests  $\rightarrow$  repository  $\rightarrow$  model  $\rightarrow$  human distributes proposal, execution, checking, persistent state, and final judgment across non-equivalent components.

## 10.4 Reliability, verification, and qualified analogies

For a path with conditional node/call reliabilities  $r_t$ ,  $\prod_t r_t$  is only an independent-error baseline. Real systems contain correlated failures, retries, redundancy, self-correction, and escalation. A *verification topology* can improve system reliability by routing generated code through a compiler and tests, a proof through a theorem checker, an API request through a schema validator, arithmetic through a calculator, or high-risk output through a human. Verification adds calls and latency, and is only useful when checks cover the relevant failure modes.

Classical distributed systems supply helpful analogies—message passing, state, scheduling, latency, failure, retries, consistency, and resource allocation—but learned nodes are distribution-sensitive, semantic interfaces may be underspecified, and decomposition/routing may itself be inferred. Likewise, multi-agent LLM systems are the homogeneous special case  $N_i = \text{LLM}_i$ ; adding agents does not by itself create a stronger class. Node semantics and communication topology determine the change. Distributed problem solving and multi-agent research provide the established computational context for such coordination [22]; Paper 0.1 contributes a connection to learned acquisition frontiers, tool algebra, reliability, and measured closure, not the broad novelty claim that cognition can be distributed.

The practical unit of analysis may therefore be a heterogeneous computational network rather than an isolated model. That perspective remains downstream of the core theory: first characterize local machines and interfaces; then ask which compositions the graph can acquire, route, verify, and execute under finite resources.

## 11 Empirical anchors from the series

These anchors occur after the framework because none is needed for its definitions or propositions. They are bounded observations, not universal claims.

**Paper 0.5: one-pass refinement.** Across three controlled generators, depth improves predictive-class organization, usually non-monotonically. Signal grows faster than fluctuation; absolute margin variance does not contract. Local-attention reachability is necessary but insufficient, and head effects are distributed and non-additive. This anchors Axes 1–2 while falsifying simple monotone-depth, contraction, and stable-head-role stories.

**Paper 0.6: competence and allocation.** Bounded structured attribute composition is a positive generator-specific result, but the original depth-general predicate phase passes no three-seed gate. A later capacity diagnostic finds that depth 8 makes `isAncestor` seed-stable on easy and medium tiers, but not hard; four other predicates remain far below competence. At twice the training budget, the complete baseline also passes easy and medium while failing hard, so optimization moves the boundary and weakens a depth-only explanation. Depth eight is near-perfect at 2x for two observed seeds, but its third seed is missing and the central gate is unevaluable. This remains a finite membership frontier, not general predicate closure.

**Paper 0.8: information versus acquisition.** The D4 permutation-completion task is information-sufficient: under its declared prior, context reduces target uncertainty from  $\log_2 3$  to zero. Yet none of five Pareto-small candidates passes all three seeds. In contextual-copy calibration, one competent checkpoint exhibits pair binding and later value routing, while fresh replications miss the 95% three-seed gate. At 1,024 episodes, seeds 11 and 23 reach 100%, whereas seed 37 moves from 52.1% at nominal budget to 81.2% at  $2\times$  and 60.4% at a targeted  $4\times$  run. This directly motivates separating information sufficiency, physical mechanism after acquisition, and reliable acquisition.

**Papers 0.7, 0.85, and 0.9: pending gates.** Paper 0.7 now has exact CPU-validated propositional and unification generators, but no formal-model frontier is yet a finding. Paper 0.85 will compare one-token and unconstrained multi-token machines; Paper 0.9 will compare scratchpad, proto-tool, and harness transitions. Their planned comparisons are hypotheses, not empirical anchors.

## 12 Core falsifiable conjectures

- C1.** Reliable acquisition of a local operator does not imply reliable acquisition of its finite compositions.
- C2.** Many iterative task families exhibit a finite one-pass composition frontier for a fixed trained Transformer.
- C3.** Multi-token state generation substantially expands the measured composition frontier for tasks admitting a reusable local transition.
- C4.** After local competence, long-recurrence failure increasingly reflects state representation, termination, counters, and accumulated error.
- C5.** Supplying facts/rules in context separates factual memory failure from procedural failure.
- C6.** For some tasks, explicit harnesses greatly improve acquisition/execution reliability without changing what an ideal autoregressive Transformer could represent in principle.

## 13 Open problems

The framework sharpens eight questions. Which local operators have high acquisition probability at a fixed scale and distribution? Which resource interactions move one-pass composition frontiers? When does autoregressive state produce a stable train-short/test-long signature rather than a longer bounded repertoire? Which tasks genuinely require counters, multiple state variables, stacks, substitutions, or visited-state memory under a specified interface? Does context provide missing facts, choose among stored tasks, estimate parameters, or construct a transition? When does a harness enlarge the semantic machine rather than stabilize an already representable trajectory? Can empirical acquisition boundaries be connected to circuit or automata classes without discarding precision and uniformity assumptions? Finally, what generator and curriculum properties favor a reusable transition over memorized finite coverage?

## 14 Conclusion

A theory of computation for LLMs should distinguish what a model can represent, what training can reliably acquire, what one pass can compose, what autoregressive recurrence can close, what context contributes as information, and where external execution performs the task. The computational object in a modern application is not the language model alone: it is a learned controller embedded in a state architecture, supplied with information interfaces and an external instruction algebra under finite resource bounds. Classical computability characterizes what this compound machine can represent in principle. A theory of learned machines must additionally characterize which procedures training discovers, with what probability, over which complexity frontier, at what cost, and with what execution reliability.

## References

- [1] M. Hahn. Theoretical Limitations of Self-Attention in Neural Sequence Models. *TACL*, 2020.
- [2] Y. Hao, D. Angluin, and R. Frank. Formal Language Recognition by Hard Attention Transformers: Perspectives from Circuit Complexity. *TACL*, 2022.
- [3] J. Pérez, P. Barceló, and J. Marinkovic. Attention is Turing Complete. *JMLR*, 2021.
- [4] C. Yun et al. Are Transformers Universal Approximators of Sequence-to-Sequence Functions? *ICLR*, 2020.
- [5] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin. What Formal Languages Can Transformers Express? A Survey. *TACL*, 2024.
- [6] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of Thought Empowers Transformers to Solve Inherently Serial Problems. *ICLR*, 2024.
- [7] N. Joshi et al. A Theory of Learning with Autoregressive Chain of Thought. *COLT*, 2025.
- [8] A. A. Bavandpour et al. Lower Bounds for Chain-of-Thought Reasoning in Hard-Attention Transformers. *ICML*, 2025.
- [9] R. Huang, Y. Liang, and J. Yang. How Transformers Learn Regular Language Recognition: A Theoretical Study on Training Dynamics and Implicit Bias. *ICML*, 2025.
- [10] W. Merrill, A. Sabharwal, and N. A. Smith. Saturated Transformers are Constant-Depth Threshold Circuits. *TACL*, 2022.
- [11] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.
- [12] E. Karpas et al. MRKL Systems: A Modular, Neuro-Symbolic Architecture That Combines Large Language Models, External Knowledge Sources and Discrete Reasoning. arXiv:2205.00445, 2022.
- [13] T. Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.
- [14] L. Gao et al. PAL: Program-Aided Language Models. *ICML*, 2023.
- [15] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR*, 2023.
- [16] D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024.
- [17] C. Olsson et al. In-Context Learning and Induction Heads. *Transformer Circuits Thread*, 2022.
- [18] N. Elhage et al. A Mathematical Framework for Transformer Circuits. *Transformer Circuits Thread*, 2021.
- [19] K. Meng et al. Locating and Editing Factual Associations in GPT. *NeurIPS*, 2022.
- [20] E. Hutchins. *Cognition in the Wild*. MIT Press, 1995.
- [21] A. Clark and D. Chalmers. The Extended Mind. *Analysis*, 58(1):7–19, 1998.
- [22] M. Wooldridge. *An Introduction to MultiAgent Systems*, second edition. Wiley, 2009.